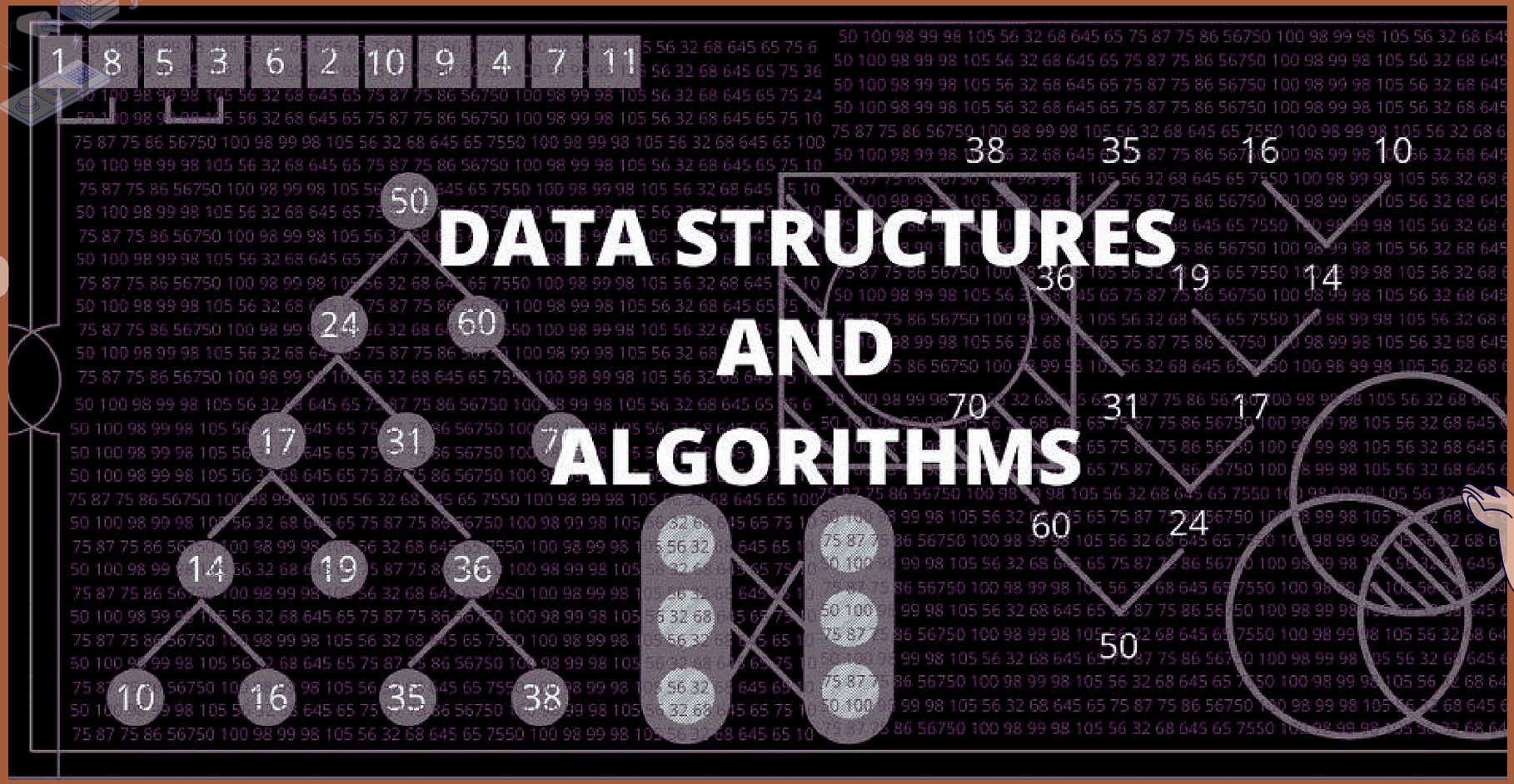
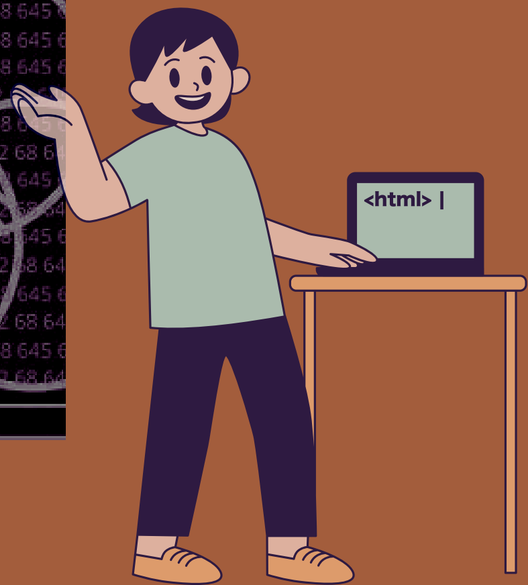
Examine abstract data types, concrete data structures and algorithms

(Chu Viet Chien - BH00915.)





The content in this process is:

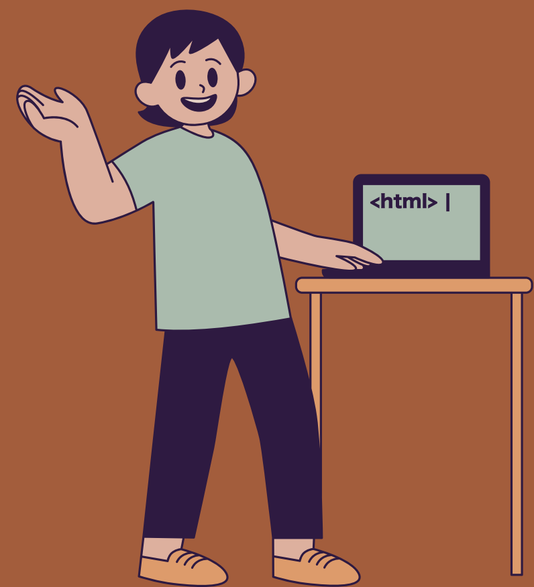
1, Create a design specification for data structures, explaining the valid operations that can be carried out on the structures.

2, Determine the operations of a memory stack and how it is used to implement function calls in a computer.

3, Illustrate, with an example, a concrete data structure for a First in First out (FIFO) queue.

4, Compare the performance of two sorting algorithms.

5, Analyse the operation, using illustrations, of two network shortest path algorithms, providing an example of each.



1, Create a design specification for data structures, explaining the valid operations that can be carried out on the structures.

1.1, Identify the Data Structures.

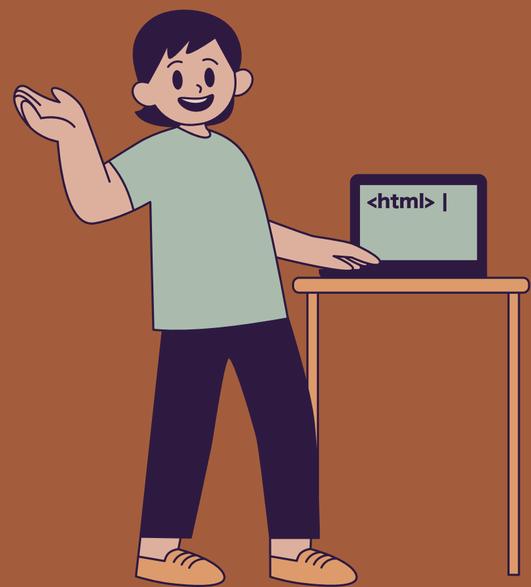
1.2, Define the Operations.

1.3, Specify Input Parameters.

1.4, Define Pre- and Post-conditions.

1.5,Discuss Time and Space Complexity.

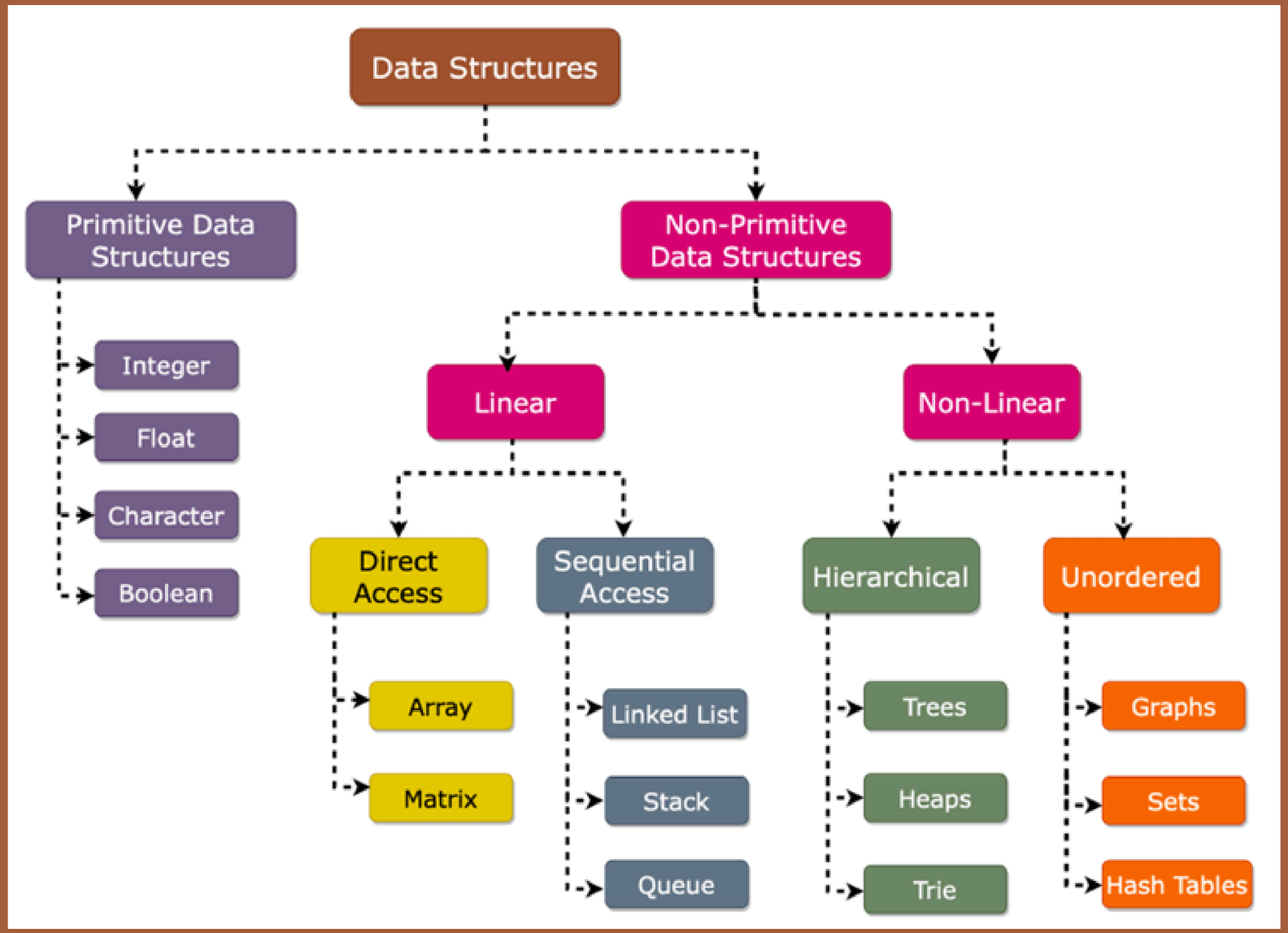
1.6, Provide Examples and Code Snippets (if applicable).



1.1, Identify the Data Structures.

1.1.1.1, What is a Data Structure?

A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. Data structures define the layout of data and the relationships between different elements, allowing for optimized operations like searching, sorting, and manipulation.



1.1.1.2, Data structure in student management application.

Data Structure	Purpose	Attributes	Key Operations
Student Class	Represents the information of each student	- `student_id` : Student ID (string/integer) - `name` : Full name (string) - `marks` : Marks (float) - `rank` : Ranking (string)	- `calculate_rank()` : Calculates and assigns rank based on marks
List/Array of Students	Stores multiple `Student` objects	- A list or array containing `Student` objects	- `Add()` : Add new student - `Delete()` : Remove student by ID/index - `Access()` : Retrieve info - `Sort()` - `Filter()`
Linked List	Dynamic structure for efficient insert/delete operations	- Each node contains a `Student` object and a pointer to the next node	- `Insert()` : Add node at a desired position - `Delete()` : Remove node - `Traverse()` : Access each node - `Search()`
Dictionary/Hash Map	Provides fast access to student records by ID	- Key-value pairs: `student_id` as key, `Student` object as value	- `Add()` : Add student - `Retrieve()` : Access student by ID - `Update()` : Modify student info - `Delete()`
Sorting Algorithms	Sorts students based on specific criteria	- Uses algorithms like Bubble Sort, Quick Sort, Merge Sort	- `Sort by Marks()` : Sort from highest to lowest marks - `Sort by Name()` : Alphabetical sorting - `Ranking()` : Assign ranks

1.2, Define the Operations.

Operation	Description	Input	Output
1, Student Class Operations.			
calculate_rank() :	Calculates and determines the rank of the student based on their marks.	Node.	Student rank: Fail (0 – 5.0), Medium (5.0 – 6.5), Good (6.5 - 7.5), Very Good (7.5 – 9.0), Excellent (9.0 – 10.0).
2, List/Array of Students Operations.			
Add :	Adds a new Student object to the list or array.	Student object containing ID, name, and marks.	Node.
Delete :	Removes a Student object from the list or array based on the student's ID or index.	Student ID or index.	Node.
Access :	Retrieves and returns the information of a Student object at a specific index.	Index of the student in the list or array.	Information of the Student object.
Sort :	Sorts the list of students based on specific attributes (marks or name).	Attribute to sort by (e.g., 'marks' or 'name').	Node.
Filter :	Extracts students that meet specific criteria.	Criteria for filtering (e.g., minimum marks).	Filtered list of students.
3, Linked List Operations.			
Insert :	Adds a new Student node at a specified position in the linked list.	Student object and position index.	Node.
Delete (Linked List) :	Removes a Student node at a specified position in the linked list.	Position index of the node to be deleted.	Node.
Traverse :	Iterates through each node in the linked list, allowing access to each Student object.	Node.	Information of each student while traversing.
Search :	Finds a Student node by traversing the linked list and checking each node's student ID.	Student ID to search.	Returns Student object if found; otherwise, not found message.
4. Dictionary/Hash Map Operations.			
Add (Dictionary) :	Adds a new Student object to the dictionary using the student ID as the key.	student_id as the key and Student object.	Node.
Retrieve :	Accesses and returns the Student object associated with a given student ID	student_id to look up.	Corresponding Student object.
Update :	Modifies the information of an existing Student object based on the student ID.	student_id and new Student object data.	Node.
Delete (Dictionary) :	Removes a Student object from the dictionary using the student ID.	student_id to identify the student to be deleted.	Node.
5. Sorting Algorithms Operations.			
Sort by Marks :	Sorts the list of students based on their marks in ascending or descending order.	Sort order (ascending or descending).	Node.
Sort by Name :	Assigns a rank to each student based on their position in the sorted list.	Sort order (ascending or descending).	Node.
Ranking :	Assigns a rank to each student based on their position in the sorted list.	Node.	List of students with ranks assigned.

1.3, Specify Input Parameters.

Operation	Input Parameters	Data Type
1. Student Class Operations		
`calculate_rank()`	- Node	- Node
2. List/Array of Students Operations		
`Add`	- `student_id` : Unique identifier for the student	- int
	- `name` : Full name of the student	- string
	- `marks` : Marks obtained by the student	- float
`Delete`	- `student_id` : Unique identifier of the student to be deleted	- int
`Access`	- `index` : Position of the student in the list or array	- int
`Sort`	- `attribute` : Attribute to sort by (e.g., 'marks', 'name')	- string
`Filter`	- `min_marks` : Minimum marks threshold for filtering students	- float
3. Linked List Operations		
`Insert`	- `student_id` : Unique identifier for the student	- int
	- `name` : Full name of the student	- string
	- `marks` : Marks obtained by the student	- float
	- `position` : Index where the student should be added	- int
`Delete (Linked List)`	- `position` : Index of the student in the linked list	- int
`Traverse`	- Node	- Node
`Search`	- `student_id` : Unique identifier of the student to search for	- int
4. Dictionary/Hash Map Operations		
`Add (Dictionary)`	- `student_id` : Unique identifier for the student (used as key)	- int
	- `name` : Full name of the student	- string
	- `marks` : Marks obtained by the student	- float
`Retrieve`	- `student_id` : Unique identifier of the student to retrieve	- int
`Update`	- `student_id` : Unique identifier for the student	- int
	- `new_name` : Updated full name	- string
	- `new_marks` : Updated marks	- float
`Delete (Dictionary)`	- `student_id` : Unique identifier of the student to delete	- int
5. Sorting Algorithms Operations		
`Sort by Marks`	- `order` : Sorting order, either ascending ('asc') or descending ('desc')	- string
`Sort by Name`	- `order` : Sorting order, either ascending ('asc') or descending ('desc')	- string
`Ranking`	- Node	- Node

1.4, Define Pre- and Post-conditions.

Operation	Pre-conditions	Post-conditions
1, Student Class Operations.		
calculate_rank() :	- Student marks must be within a valid range (0.0 - 10.0). - Marks must be already recorded for the student.	- The rank of the student is determined based on their marks. - The rank is stored or displayed appropriately (Fail, Medium, Good, Very good, Excellent).
2, List/Array of Students Operations.		
Add :	- The student_id must be unique and not already exist in the list/dictionary. - Student's name and marks must be provided.	- A new Student object is added to the list/dictionary.
Delete :	- The student_id or index must exist in the list/dictionary.	- The corresponding student entry is removed from the list/dictionary.
Access :	- The index must be valid and within the range of the list or array size.	- The student information at the specified index is returned/displayed.
Sort :	- The list of students must not be empty. - Attribute (e.g., 'marks', 'name') must be valid for sorting.	- The student list is sorted according to the specified attribute (e.g., marks or name).
Filter :	- The list of students must not be empty. - The min_marks must be within the valid range of marks (0.0 - 10.0).	- A filtered list of students who meet the criteria is returned.
3, Linked List Operations.		
Insert :	- The student_id must be unique. - Position index must be valid within the list/array range.	- A new student is inserted at the specified position in the list or array.
Delete (Linked List) :	- The list must contain students. - The position index must be valid within the list range.	- The student node at the specified position is removed from the linked list.
Traverse :	- The linked list must contain at least one student node.	- All student nodes are traversed, and information is accessed or displayed for each student.
Search :	- The list or dictionary must contain students. - The student_id must be provided and valid.	- The student with the given student_id is found and returned if present. If not found, an appropriate message is displayed.

4. Dictionary/Hash Map Operations.		
Add (Dictionary) :	- The student_id must be unique and not already exist in the dictionary. - Student's name and marks must be provided.	- A new student entry is added to the dictionary.
Retrieve :	- The student_id must exist in the dictionary.	- The corresponding student information is returned.
Update :	- The student_id must exist in the dictionary. - Updated student information (name or marks) must be provided.	- The student information is updated in the dictionary with new details.
Delete (Dictionary) :	- The student_id must exist in the dictionary.	- The student entry is removed from the dictionary.
5. Sorting Algorithms Operations.		
Sort by Marks :	- The list of students must not be empty.	- The list is sorted in ascending or descending order based on student marks.
Sort by Name :	- The list of students must not be empty.	- The list is sorted in alphabetical order by student name.
Ranking :	- The list of students must already be sorted.	- Students are assigned ranks based on their position in the sorted list.

1.5,Discuss Time and Space Complexity.

1.1.5.1, Time and space complexity table.

Operation	Time Complexity	Space Complexity	Explanation
1, Student Class Operations.			
calculate_rank() :	O(1)	O(1)	The ranking is determined directly based on the student's marks, which takes constant time and space.
2, List/Array of Students Operations.			
Add :	O(1) for list append, O(n) for inserting at an arbitrary position.	O(1) for appending, O(n) for shifting in an array-based list.	Appending a student at the end of the list takes constant time, but inserting at a specific position requires shifting elements, which takes linear time.
Delete :	O(n)	O(1)	Deleting a student requires searching for the student in the list, which takes linear time. Memory usage remains constant since only one element is deleted.
Access :	O(1)	O(1)	Accessing a student by index in a list is constant in time and space.
Sort :	O(n log n).	O(n)	Sorting a list using a comparison-based algorithm like Merge Sort or Quick Sort takes O(n log n) time. Sorting uses additional memory for temporary storage (O(n)).
Filter :	O(n)	O(k), where k is the number of filtered students.	Filtering involves iterating through the list, which is linear in time. The space required depends on the number of students who meet the filter condition.

3, Linked List Operations.			
Insert :	O(n)	O(1)	Inserting a student at a specific position requires shifting other students, which takes linear time. However, memory usage remains constant.
Delete (Linked List) :	O(n)	O(1)	Deleting from a linked list involves traversing the list to find the target node (O(n)), but no extra memory is required (constant space).
Traverse :	O(n)	O(1)	Traversing the list involves visiting each node once, which takes linear time. No additional space is required during traversal.
Search :	O(n)	O(1)	Searching for a student in a list requires iterating through the entire list, resulting in linear time complexity. The memory usage is constant.

4. Dictionary/Hash Map Operations.			
Add (Dictionary) :	O(1) on average.	O(1)	Adding an element to a dictionary (hash table) takes constant time on average, and the space required grows linearly with the number of entries.
Retrieve :	O(1) on average.	O(1)	Retrieving an element from a dictionary takes constant time and space on average.
Update :	O(1) on average.	O(1)	Updating a dictionary value takes constant time and does not require additional space.
Delete (Dictionary) :	O(1) on average.	O(1)	Deleting from a dictionary also takes constant time on average, with constant space usage.

5. Sorting Algorithms Operations.			
Sort by Marks :	O(n log n).	O(n)	Sorting by marks is the same as general sorting. Sorting a list of size n takes O(n log n) time with extra space of O(n) for temporary storage.
Sort by Name :	O(n log n).	O(n)	Sorting by name also takes O(n log n) time with O(n) space for storage.
Ranking :	O(n)	O(1)	Assigning ranks after sorting requires a single pass through the list (O(n)). The memory usage is constant since ranks are assigned directly to students.

1.1.5.2, Explanation.

a, Time Complexity.

- **O(1) (Constant Time): Some operations, such as accessing an element by index, retrieving or updating a dictionary value, and calculating a student's rank based on marks, take constant time regardless of the number of students.**
- **O(n) (Linear Time): Operations like searching for a student, filtering, traversing, and inserting or deleting in the middle of a list require linear time as they involve processing each student once.**
- **O(n log n): Sorting operations are typically more complex, requiring O(n log n) time, which is efficient but more expensive than linear time.**

b, Space Complexity.

- **O(1) (Constant Space): Many operations, such as accessing, deleting, or updating students, do not require additional memory beyond the space already used by the data structure.**
- **O(n) (Linear Space): Sorting and filtering may require additional memory to store intermediate results or temporary arrays.**

1.6, Provide Examples and Code Snippets (if applicable).

1.6.1, Stack implementation code in java.

SDL > Node.java

```java
package org.example;

public class Node {
    int data; // Data stored in the node
    Node next; // Pointer to the next node

    Node(int data) {
        this.data = data; // Initialize the node with data
        this.next = null; // Set next to null
    }
}
```

SDL > Stack.java

```java
package org.example;

public class Stack {
    private Node top; // Pointer to the top node
    private int size; // Size of the stack

    public Stack() {
        this.top = null; // Initialize top as null
        this.size = 0; // Initialize size as 0
    }

    // Push operation
    public void push(int element) {
        Node newNode = new Node(element); // Create a new node
        newNode.next = top; // Link new node to the top
        top = newNode; // Update top to the new node
        size++; // Increase size
    }

    // Pop operation
    public int pop() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty"); // Handle empty stack
        }
        int poppedData = top.data; // Get data from top node
        top = top.next; // Move top to next node
        size--; // Decrease size
        return poppedData; // Return popped data
    }

    // Peek operation
    public int peek() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty"); // Handle empty stack
        }
        return top.data; // Return data from top node
    }

    // Check if the stack is empty
    public boolean isEmpty() {
        return size == 0; // Return true if size is 0
    }

    // Get size of the stack
    public int size() {
        return size; // Return current size
    }
}
```

1.6.2, input.

SDL > Main.java

```java
package org.example;

public class Main {
    public static void main(String[] args) {
        Stack stack = new Stack(); // Create a new stack

        // Push elements onto the stack
        stack.push(10);
        stack.push(20);
        stack.push(30);

        // Display the size of the stack
        System.out.println("Size of stack: " + stack.size()); // Output: 3

        // Peek at the top element
        System.out.println("Top element: " + stack.peek()); // Output: 30

        // Pop elements from the stack
        System.out.println("Popped: " + stack.pop()); // Output: 30
        System.out.println("Popped: " + stack.pop()); // Output: 20
        System.out.println("Popped: " + stack.pop()); // Output: 10

        // Check if stack is empty
        System.out.println("Is stack empty? " + stack.isEmpty()); // Output: true

    }
}
```

1.6.3, output.

```
C:\Users\ADMIN\.jdks\corretto-17.0.12\bir
Size of stack: 3
Top element: 30
Popped: 30
Popped: 20
Popped: 10
Is stack empty? true

Process finished with exit code 0
```

2, Determine the operations of a memory stack and how it is used to implement function calls in a computer.

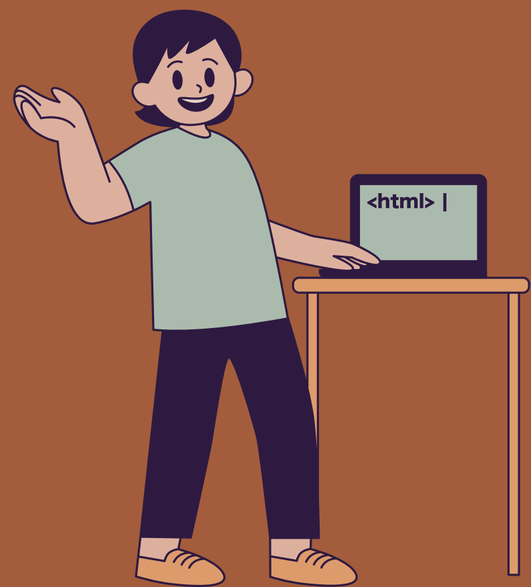
2.1, Define a Memory Stack.

2.2, Identify Operations.

2.3, Function Call Implementation.

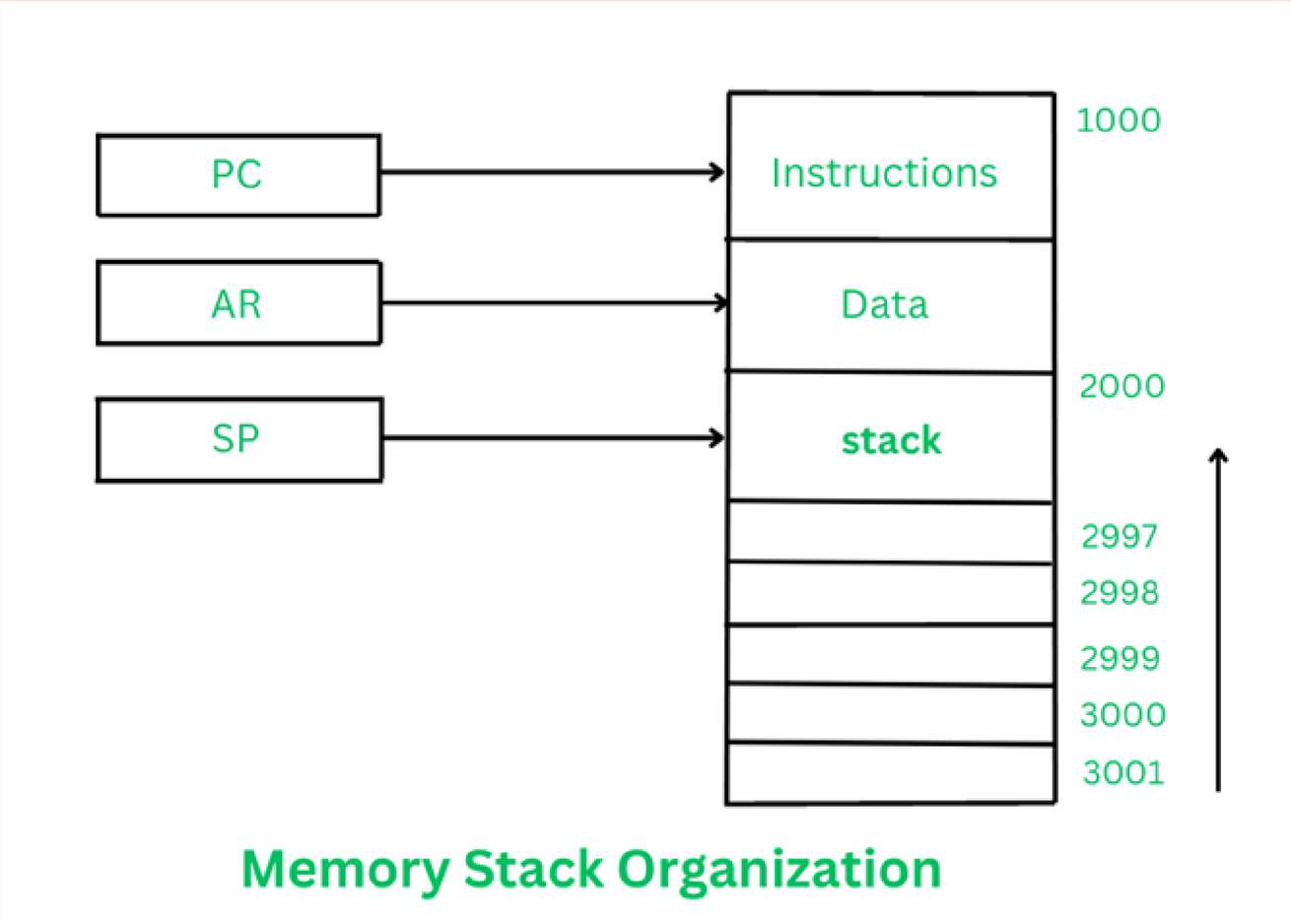
2.4, Demonstrate Stack Frames.

2.5,Discuss the Importance.



2.1, Define a Memory Stack.

A memory stack is a data structure used in computer systems for managing function calls, local variables, and control flow during program execution. It operates on a Last-In, First-Out (LIFO) principle, meaning the last element added is the first to be removed. In the context of memory, the stack stores activation records or stack frames, which hold information needed for the execution of functions (like parameters, return addresses, and local variables).



2.2, Identify Operations.

1.2.2.1, Operations of a Memory Stack.

Operation	Description
Push :	- Adds a new element (e.g., function call, local variable, or return address) to the top of the stack. This happens when a function is called.
Pop :	- Removes the top element from the stack. This occurs when a function finishes execution, and its stack frame is no longer needed.
Peek (Top) :	- Returns the value of the top element without removing it from the stack. Useful for inspecting the most recent data (e.g., return address) on the stack.
IsEmpty :	- Checks whether the stack is empty. Indicates that no function calls are active if the stack is empty.
IsFull:	- Checks whether the stack is full. In modern systems, this is rare, but it can occur in cases of deep recursion or constrained memory environments.

1.2.2.2, The operation of the stack in the student management application.

Situation	Stack Operation	Description	Example
Adding a Student :	Push	Save the current state of the student list before adding a new student.	Before adding a student with ID 001, save the current list to undo if necessary.
Editing Student Information :	Push	Save the old information of the student before modifying it.	When editing the information of a student with ID 001, save the old information for recovery if needed.
Deleting a Student :	Push	Store the student's information before deleting it for potential recovery later.	Before deleting a student with ID 001, save that student's information onto the stack for later undoing.
Undo :	Pop	Retrieve information from the stack to restore the state before performing an add, edit, or delete operation.	If a new student was just added and you want to undo, the student's information will be popped from the stack.
Redo :	Push & Pop	Revert back to the state after an undo, restoring the previously undone data to continue the operation.	After undoing the addition of a new student, if you want to redo, the student's information will be pushed back onto the stack.
Browsing the List :	Push (when going back) & Pop (when returning).	Go backwards through the student list, saving the current student's information onto the stack and displaying the previous student.	If viewing information for student 002 and want to return to student 001, save student 002's info to the stack and display student 001.
Memory Optimization :	Push & Pop	Only store necessary information, retrieving data when needed to optimize space and processing time.	Only store information about students during management operations, saving memory and processing time.
Error Management:	Push	Save information about errors occurring during operations with the student list for processing or undoing.	If an error occurs while editing student information, details of the error will be stored in the stack for later review.

Explanation of Stack Operations:

- **Push: Saves information onto the stack when performing operations like add, edit, or delete. This allows for recovery of previous data when needed.**
- **Pop: Retrieves information from the stack to undo operations or restore previous states. This action helps revert the system back to its state before changes were made.**
- **Browsing the List: When the user wants to view information about a previous student, the current information will be saved onto the stack, allowing for easy return.**
- **Memory Optimization: The stack only stores necessary information, which helps save memory and speeds up processing as only important data is kept.**
- **Error Management: When an error occurs, information about the error will be saved to the stack for easy checking and processing later.**

2.3, Function Call Implementation.

1.2.3.1, Stack Structure.

A stack operates on a Last In, First Out (LIFO) principle, meaning the last function that is called will be the first one to complete execution and return control to the calling function. Each function call creates a new stack frame that holds the function's execution context.

1.2.3.2, Components of a Stack Frame.

A stack frame typically contains:

- Return Address: The location in the code to return to after the function completes.
- Parameters: The input values passed to the function.
- Local Variables: Variables declared within the function.
- Saved Registers: Values of CPU registers that need to be preserved for the calling function.

1.2.3.3, Function Call Process.

When a function is called, the following steps occur:

- Push Parameters: The parameters of the function are pushed onto the stack.
- Push Return Address: The address of the next instruction to execute after the function call is pushed onto the stack.
- Create Stack Frame: A new stack frame is created for the called function. This includes space for local variables and the saved registers.
- Execute Function: The function executes using the parameters and local variables stored in the stack frame.
- Return Value: If the function returns a value, it is typically placed in a specific register (like eax in x86 architecture) or at a specific memory location.
- Pop Stack Frame: Once the function completes, the stack frame is popped from the stack, restoring the previous state of the stack.
- Pop Return Address: The return address is popped from the stack, and control is transferred back to this address.

1.2.3.4, Example of Function Call Implementation.

a, Input.

```java
public class Student {
    int add(int a, int b) {
        int sum = a * b; // Local variable
        return sum;      // Return value
    }

    public static void main(String[] args) {
        Student student = new Student();   // Object creation
        int result = student.add(5, 3);    // Function call
        System.out.println(result);        // Print the result
    }
}
```

b, output.

```
C:\Users\ADMIN\.jdks\corretto-17.0.12\bin
15

Process finished with exit code 0
```

Explanation of Function Call Implementation:

a, Class Declaration.

- The class Student is declared, which contains methods that can be used to perform operations related to students.

b, Method Definition (add):

- The add method is defined to take two integer parameters, a and b.
- Inside the method, it calculates the sum by adding a and b and stores it in a local variable sum.
- The method then returns the sum to the caller.

c, Main Method:

- The main method serves as the entry point of the program.
- An instance of the Student class is created to access the instance method add.
- The add method is called with 5 and 3 as arguments, and the result is stored in the variable result.
- Finally, the result is printed out to the dashboard (here the result is 15).

2.4, Demonstrate Stack Frames.

What is a Stack Frame?

A **stack frame** is a data structure that contains the state of a single method call. This includes:

- **Arguments**: Parameters passed to the method.
- **Local Variables**: Variables declared inside the method.
- **Return Address**: The point where the program should return after the method finishes.
- **Intermediate Calculations**: If there are any, during the method execution.

1.2.3.4, Example of Function Call Implementation.

a, Input.

```
public class Student {
    int add(int a, int b) {
        int sum = a * b; // Local variable
        return sum;      // Return value
    }

    public static void main(String[] args) {
        Student student = new Student();   // Object creation
        int result = student.add(5, 3);    // Function call
        System.out.println(result);        // Print the result
    }
}
```

b, output.

```
C:\Users\ADMIN\.jdks\corretto-17.0.12\bin
15

Process finished with exit code 0
```

Stack Frame Visualization:

At different points during execution, the stack looks like this:

1. **Main Method Start**:

```csharp
[main method frame]
```

2. **Calling** `add(5, 3)`:

```sql
[add method frame: a=5, b=3, sum=undefined]
[main method frame]
```

3. **Inside** `add` **(after calculation)**:

```sql
[add method frame: a=5, b=3, sum=15]
[main method frame]
```

4. **Returning from** `add`:

```csharp
[main method frame]
```

5. **After Program Ends**:

```php
(empty)
```

2.5,Discuss the Importance.

The memory stack plays a critical role in a computer's execution of programs, particularly in:

- **Function Call Management**: The stack ensures that function calls can be nested and recursive, maintaining the correct execution flow.
- **Memory Efficiency**: Local variables and function call information are only stored as long as the function is active. Once the function completes, its frame is removed from the stack, freeing memory.
- **Recursion Handling**: The stack allows the system to handle recursive function calls by maintaining separate stack frames for each call, enabling each recursive call to have its own scope.
- **Control Flow Integrity**: The stack ensures that functions return to the correct location in the program by storing the return address in the stack frame.

3, Illustrate, with an example, a concrete data structure for a First in First out (FIFO) queue.

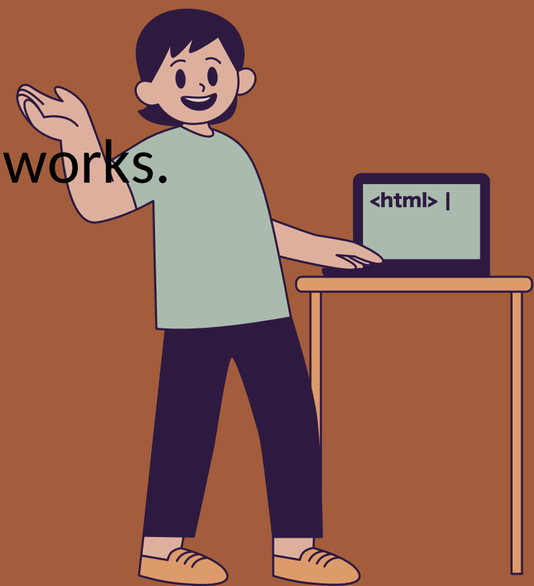
3.1,Introduction FIFO.

3.2, Define the Structure.

3.3, Array-Based Implementation.

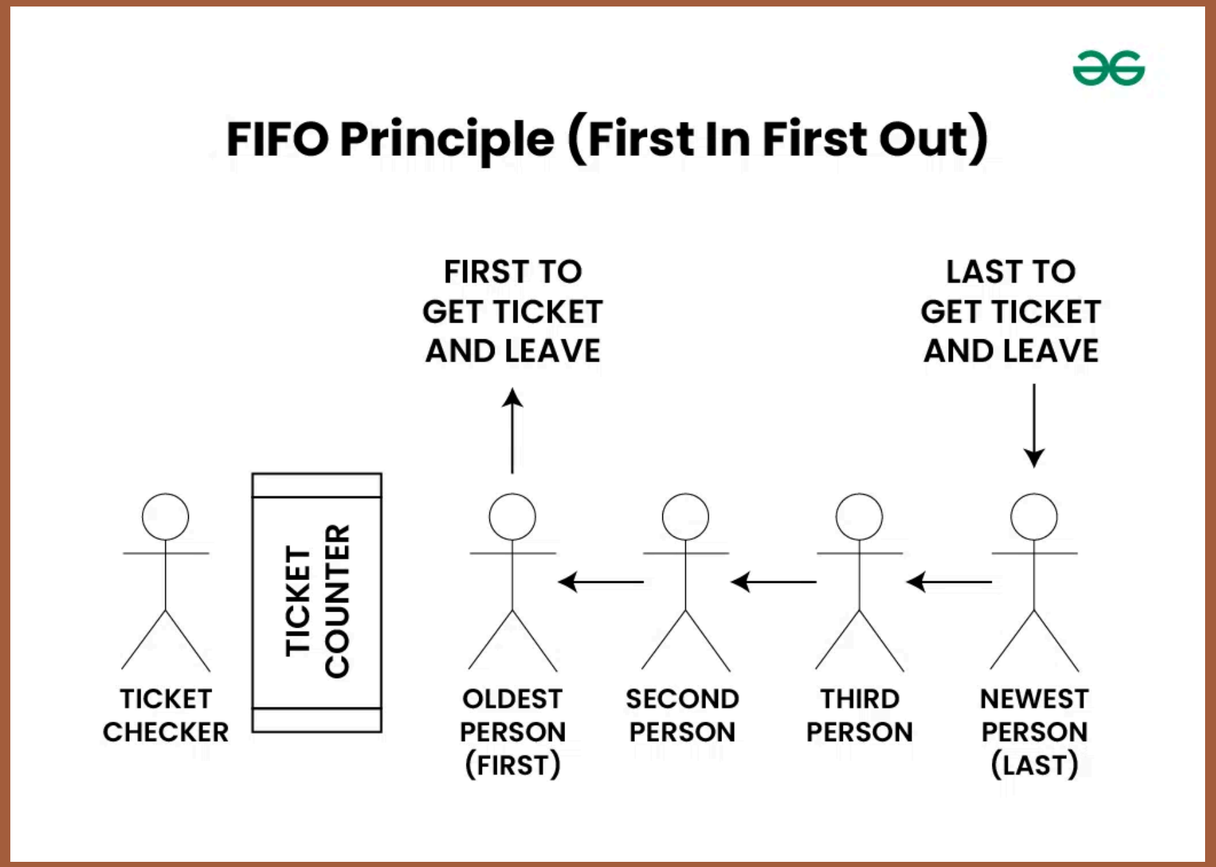
3.4, Linked List-Based Implementation.

3.5, Provide a concrete example to illustrate how the FIFO queue works.



3.1,Introduction FIFO.

A FIFO queue is a data structure that follows the First In, First Out principle, meaning that the first element added to the queue will be the first one to be removed. This is similar to a real-life queue, such as waiting in line at a store: the person who enters the line first is served first. Queues are widely used in computer science, particularly in scenarios such as task scheduling, buffering, and handling requests.



3.2, Define the Structure.

A queue typically supports the following operations:

- **Enqueue (add)**: Add an element to the end (tail) of the queue.
- **Dequeue (remove)**: Remove an element from the front (head) of the queue.
- **Peek (front)**: View the front element without removing it.
- **isEmpty**: Check if the queue is empty.
- **isFull** (for fixed-size queues): Check if the queue is full.

3.3, Array-Based Implementation.

SDL > ArrayQueue.java

```java
package org.example;

public class ArrayQueue {
    private int[] queue;
    private int front, rear, size, capacity;

    // Constructor to initialize the queue
    public ArrayQueue(int capacity) {
        this.capacity = capacity;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    // Check if the queue is full
    public boolean isFull() {
        return size == capacity;
    }

    // Check if the queue is empty
    public boolean isEmpty() {
        return size == 0;
    }

    // Enqueue operation
    public void enqueue(int item) {
        if (isFull()) {
            System.out.println("Queue is full.");
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = item;
        size++;
        System.out.println("Enqueued: " + item);
    }

    // Dequeue operation
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue is empty.");
            return -1;
        }
        int item = queue[front];
        front = (front + 1) % capacity;
        size--;
        System.out.println("Dequeued: " + item);
        return item;
    }
```

3.4, Linked List-Based Implementation.

SDL > Node.java

```java
package org.example;

public class Node {
    int data; // Data stored in the node
    Node next; // Pointer to the next node

    Node(int data) {
        this.data = data; // Initialize the node with data
        this.next = null; // Set next to null
    }
}
```

SDL > LinkedListQueue.java

```java
package org.example;

public class LinkedListQueue {
    private Node front, rear;

    // Constructor to initialize the queue
    public LinkedListQueue() {
        this.front = this.rear = null;
    }

    // Check if the queue is empty
    public boolean isEmpty() {
        return front == null;
    }
```

```java
    // Enqueue operation
    public void enqueue(int item) {
        Node newNode = new Node(item);
        if (this.rear == null) {
            this.front = this.rear = newNode;
            System.out.println("Enqueued: " + item);
            return;
        }
        this.rear.next = newNode;
        this.rear = newNode;
        System.out.println("Enqueued: " + item);
    }
    // Dequeue operation
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue is empty.");
            return -1;
        }
        int item = front.data;
        front = front.next;

        if (front == null) {
            rear = null;
        }
        System.out.println("Dequeued: " + item);
        return item;
    }
    // Peek operation
    public int peek() {
        if (isEmpty()) {
            System.out.println("Queue is empty.");
            return -1;
        }
        return front.data;
    }
}
```

3.5, Provide a concrete example to illustrate how the FIFO queue works.

3.5.1, input.

SDL > Main2.java

```java
package org.example;

public class Main2 {
    public static void main(String[] args) {
        ArrayQueue queue = new ArrayQueue(5);

        queue.enqueue(10);
        queue.enqueue(20);
        queue.enqueue(30);
        queue.enqueue(40);

        System.out.println("Front element is: " + queue.peek()); // Output: 10

        queue.dequeue();  // Removes 10
        queue.dequeue();  // Removes 20

        queue.enqueue(50);
        queue.enqueue(60); // Circular array example

        System.out.println("Front element is: " + queue.peek()); // Output: 30
    }
}
```

3.5.2, output.

```
C:\Users\ADMIN\.jdks\corretto-17.0.12\bin\java.exe "-
Enqueued: 10
Enqueued: 20
Enqueued: 30
Enqueued: 40
Front element is: 10
Dequeued: 10
Dequeued: 20
Enqueued: 50
Enqueued: 60
Front element is: 30

Process finished with exit code 0
```

4, Compare the performance of two sorting algorithms.

4.1, Introducing the two sorting algorithms you will be comparing.

4.2, Time Complexity Analysis.

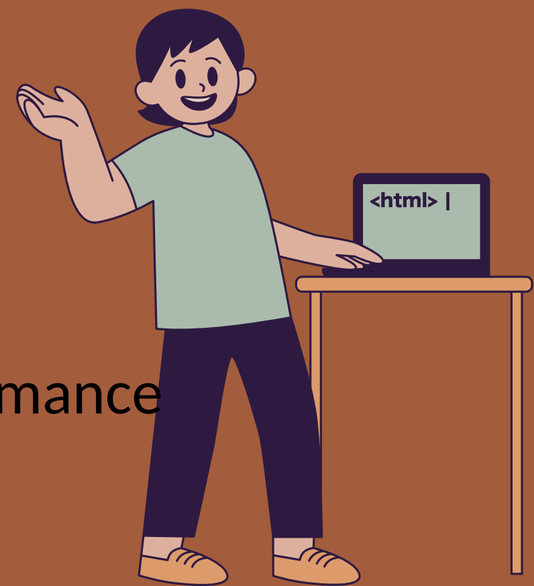
4.3, Space Complexity Analysis.

4.4, Stability.

4.5, Comparison Table.

4.6, Performance Comparison.

4.7, Provide a concrete example to demonstrate the differences in performance between the two algorithms.



4.1, Introducing the two sorting algorithms you will be comparing.

1. **Quick Sort**:
   - Quick Sort is a divide-and-conquer algorithm that selects a 'pivot' element from the array and partitions the other elements into two sub-arrays according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.
   - It is typically implemented in-place, meaning it requires minimal additional space.
2. **Merge Sort**:
   - Merge Sort is also a divide-and-conquer algorithm that divides the array into two halves, sorts each half recursively, and then merges the sorted halves back together.
   - Unlike Quick Sort, Merge Sort requires additional space for temporary arrays to hold the merged results.

4.2, Time Complexity Analysis.

- Quick Sort:
  - **Best Case**: $O(n \log n)$ when the pivot divides the array into two equal halves.
  - **Average Case**: $O(n \log n)$.
  - **Worst Case**: $O(n^2)$ when the pivot is the smallest or largest element (e.g., when the array is already sorted).
- Merge Sort:
  - **Best Case**: $O(n \log n)$.
  - **Average Case**: $O(n \log n)$.
  - **Worst Case**: $O(n \log n)$ because it always divides the array in half and merges, regardless of the initial order of the elements.

4.3, Space Complexity Analysis.

- Quick Sort:
  - **Space Complexity**: $O(\log n)$ for the recursive call stack. It can be implemented in place, leading to lower memory usage.
- Merge Sort:
  - **Space Complexity**: $O(n)$ due to the temporary arrays used for merging. This can be a disadvantage when working with large datasets.

4.4, Stability.

- Quick Sort:
  - **Stability**: Quick Sort is not a stable sort. Equal elements may not retain their relative order after sorting.
- Merge Sort:
  - **Stability**: Merge Sort is a stable sort. Equal elements retain their relative order, which can be important in certain applications.

4.5, Comparison Table.

Feature	Quick Sort	Merge Sort
Time Complexity (Best)	$O(n \log n)$	$O(n \log n)$
Time Complexity (Avg)	$O(n \log n)$	$O(n \log n)$
Time Complexity (Worst)	$O(n^2)$	$O(n \log n)$
Space Complexity	$O(\log n)$	$O(n)$
Stability	Not stable	Stable

4.6, Performance Comparison.

In practice, Quick Sort is generally faster than Merge Sort for average-case scenarios due to its lower constant factors and in-place nature. However, Merge Sort's guaranteed $O(n \log n)$ performance makes it preferable for large datasets, particularly when stability is required.

4.7, Provide a concrete example to demonstrate the differences in performance between the two algorithms.

Consider the following array that needs to be sorted:

Array: $[38, 27, 43, 3, 9, 82, 10]$

1. **Quick Sort**:
   - Choose a pivot (e.g., 38).
   - Partition: $[27, 3, 9, 10] | 38 | [43, 82]$
   - Recursively apply Quick Sort to the left and right sub-arrays.
   - Result: $[3, 9, 10, 27, 38, 43, 82]$
2. **Merge Sort**:
   - Split: $[38, 27, 43] | [3, 9, 82, 10]$
   - Recursively sort each half:
     - Left: $[27, 38, 43]$
     - Right: $[3, 9, 10, 82]$
   - Merge the sorted halves: $[3, 9, 10, 27, 38, 43, 82]$

5, Analyse the operation, using illustrations, of two network shortest path algorithms, providing an example of each.

5.1, Introducing the concept of network shortest path algorithms.

5.2, Algorithm 1: Dijkstra's Algorithm.

5.3, Algorithm 2: Prim-Jarnik Algorithm.

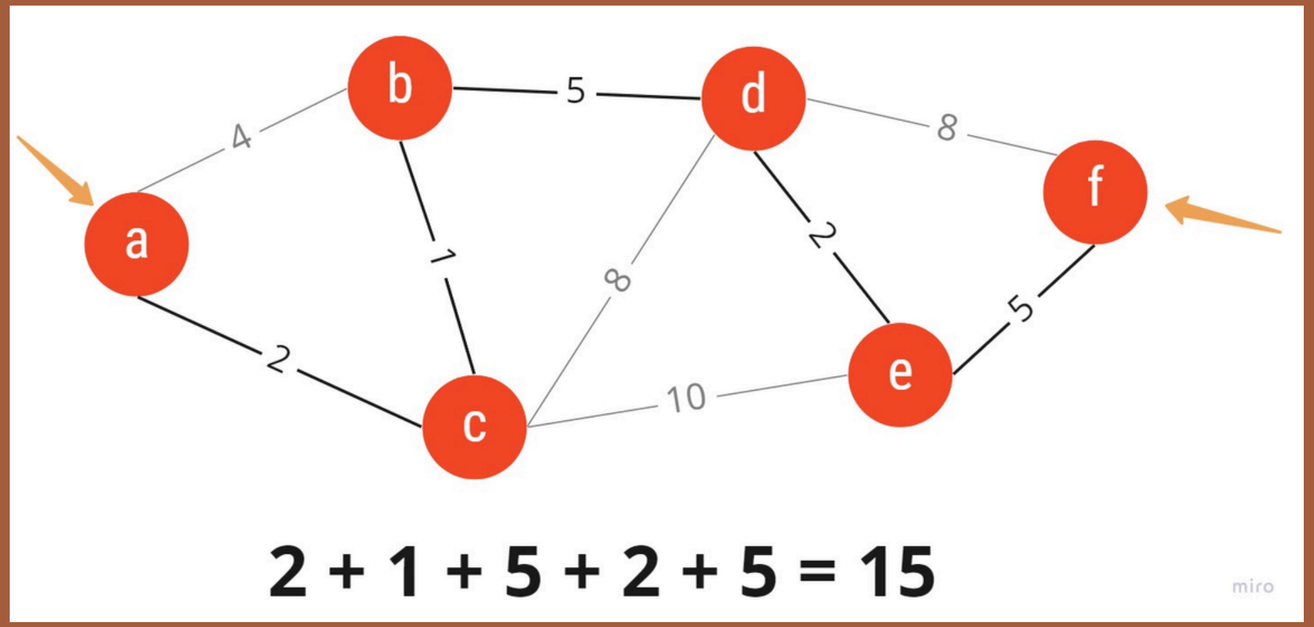
5.4, Performance Analysis.

5.1, Introducing the concept of network shortest path algorithms.

Network shortest path algorithms are designed to find the most efficient route from a source node to a destination node within a weighted graph or network. These algorithms play a crucial role in various applications, including routing in computer networks, transportation systems, and logistics. The objective is to minimize the total weight (or cost) of the path, which can represent distance, time, or any other metric.

Key Characteristics of Shortest Path Algorithms:

- Graph Representation: Networks are typically represented as graphs, with nodes (vertices) and weighted edges (links).
- Optimal Path: The algorithms aim to find the shortest (or least costly) path based on the weights of the edges.
- Dynamic and Static: Some algorithms are designed for static graphs (fixed edges and weights), while others can adapt to dynamic changes in the graph.



5.2, Algorithm 1: Dijkstra's Algorithm.

Dijkstra's Algorithm is one of the most well-known algorithms for finding the shortest path in a graph with non-negative edge weights.

Operation of Dijkstra's Algorithm:

1. **Initialization**:
   - Set the distance to the source node to zero and all other nodes to infinity.
   - Mark all nodes as unvisited. Use a priority queue (or a min-heap) to efficiently retrieve the node with the smallest distance.
2. **Iteration**:
   - While there are unvisited nodes:
     - Select the unvisited node with the smallest distance (let’s call it the "current node").
     - For each neighbor of the current node, calculate the tentative distance through the current node.
     - If the tentative distance is less than the currently recorded distance, update the distance.
3. **Completion**:
   - Mark the current node as visited. Repeat until all nodes have been visited or the smallest tentative distance among the unvisited nodes is infinity.

Example of Dijkstra's Algorithm:

Consider the following weighted graph:

```mathematica
      2        1
  A ---- B ---- C
  |      |      |
4 |    3 |      | 2
  |      |      |
  D ---- E ---- F
      1        3
```

- **Source Node**: A
- **Distances**: Initial distances are set as follows:
  - A: 0
  - B: ∞
  - C: ∞
  - D: ∞
  - E: ∞
  - F: ∞

Steps:

1. Start at A:
   - Update distances to B (2) and D (4).
2. Move to B (smallest distance):
   - Update distances to C (3 = 2 + 1) and E (5 = 2 + 3).
3. Move to D:
   - Update distance to E (5 remains).
4. Move to C:
   - Update distance to F (5 = 3 + 2).
5. Move to E:
   - F remains at 5.
6. Move to F, completing the process.

Final Distances:

- A: 0
- B: 2
- C: 3
- D: 4
- E: 5
- F: 5

5.3, Algorithm 2: Prim-Jarnik Algorithm.

Prim-Jarnik Algorithm is a greedy algorithm used primarily to find the Minimum Spanning Tree (MST) of a weighted graph, which can also be adapted to find shortest paths in certain contexts.

Operation of Prim-Jarnik Algorithm:

1. **Initialization**:
   - Start with a selected node (arbitrarily chosen) and mark it as part of the MST.
   - Initialize a priority queue to hold edges, sorted by weight.
2. **Iteration**:
   - While the MST has fewer than $N - 1$ edges (where N is the number of nodes):
     - Extract the minimum weight edge from the priority queue.
     - If the edge connects to a node not yet in the MST, add the edge to the MST and mark the new node as part of the MST.
     - Add all edges connecting to the new node to the priority queue.
3. **Completion**:
   - The process continues until all nodes are included in the MST.

Example of Prim-Jarnik Algorithm:

Consider the same graph as above:

```mathematica
      2       1
  A ---- B ---- C
  |      |      |
4|     3|      |2
  |      |      |
  D ---- E ---- F
      1       3
```

Steps:

1. Start at A. Add edges (A-B, A-D) to the priority queue.
2. Select the edge with the smallest weight (A-B, weight 2). Add B to MST.
3. Add edges connecting B (B-C and B-E) to the queue.
4. Select the next smallest edge (B-C, weight 1). Add C to MST.
5. Continue selecting edges (D-E, weight 1) until all nodes are included.

Final MST Edges:

- A-B (2)
- B-C (1)
- A-D (4)
- D-E (1)

5.4, Performance Analysis.

Time Complexity:

- **Dijkstra's Algorithm**:
  - Using a binary heap: $O((V + E) \log V)$, where V is the number of vertices and E is the number of edges.
- **Prim-Jarnik Algorithm**:
  - Using a binary heap: $O((V + E) \log V)$.

Space Complexity:

- Both algorithms typically require $O(V)$ space for storing distances and priority queues.

Stability and Applicability:

- Dijkstra's Algorithm is efficient for graphs with non-negative weights and is commonly used in network routing.
- Prim-Jarnik Algorithm is primarily used for constructing Minimum Spanning Trees but can find shortest paths in certain cases (especially when graph edges represent connection costs).